

ANSI SQL USING MYSQL

1.Show a list of all upcoming events a user is registered for in their city, sorted by date.

```
SELECT
    u.full_name,
    e.title,
    e.city,
    e.start_date
FROM Users u
JOIN Registrations r
    ON u.user_id = r.user_id
JOIN Events e
    ON r.event_id = e.event_id
WHERE e.status = 'upcoming'
    AND u.city = e.city
ORDER BY e.start_date;
```

2. Identify events with the highest average rating, considering only those that have received at least 10 feedback submissions.

```
SELECT
    e.event_id,
    e.title,
    AVG(f.rating) AS avg_rating
FROM Events e
JOIN Feedback f
    ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(f.feedback_id) >= 10
ORDER BY avg_rating DESC;
```

3.Retrieve users who have not registered for any events in the last 90 days.

```
SELECT *
FROM Users
WHERE user_id NOT IN (
    SELECT DISTINCT user_id
    FROM Registrations
    WHERE start_date >= DATE_SUB(CURDATE(), INTERVAL 90 DAY);
```

```
WHERE registration_date >= CURDATE() - INTERVAL 90 DAY  
);
```

4. Count how many sessions are scheduled between 10 AM to 12 PM for each event.

```
SELECT e.title, COUNT(s.session_id) AS session_count  
FROM Events e  
LEFT JOIN Sessions s  
ON e.event_id = s.event_id  
AND TIME(s.start_time) BETWEEN '10:00:00' AND '12:00:00'  
GROUP BY e.event_id, e.title;
```

5. List the top 5 cities with the highest number of distinct user registrations.

```
SELECT u.city, COUNT(DISTINCT r.user_id) AS registrations  
FROM Users u  
JOIN Registrations r ON u.user_id = r.user_id  
GROUP BY u.city  
ORDER BY registrations DESC  
LIMIT 5;
```

6. Generate a report showing the number of resources uploaded for each event.

```
SELECT e.title, COUNT(r.resource_id) AS total_resources  
FROM Events e  
LEFT JOIN Resources r ON e.event_id = r.event_id  
GROUP BY e.event_id, e.title;
```

7. List all users who gave feedback with a rating less than 3.

```
SELECT u.full_name, e.title, f.comments, f.rating  
FROM Feedback f  
JOIN Users u ON f.user_id = u.user_id  
JOIN Events e ON f.event_id = e.event_id  
WHERE f.rating < 3;
```

8. Display all upcoming events with the count of sessions scheduled for them.

```
SELECT e.title, COUNT(s.session_id) AS total_sessions  
FROM Events e  
LEFT JOIN Sessions s ON e.event_id = s.event_id  
WHERE e.status = 'upcoming'  
GROUP BY e.event_id, e.title;
```

9. For each event organizer, show the number of events created and their current status.

```
SELECT u.full_name, e.status, COUNT(*) AS total_events
FROM Events e
JOIN Users u ON e.organizer_id = u.user_id
GROUP BY u.full_name, e.status;
```

10. Identify events that had registrations but received no feedback at all.

```
SELECT e.title
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
LEFT JOIN Feedback f ON e.event_id = f.event_id
WHERE f.feedback_id IS NULL
GROUP BY e.event_id, e.title;
```

11. Find the number of users who registered each day in the last 7 days.

```
SELECT registration_date, COUNT(*) AS users_count
FROM Users
WHERE registration_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY registration_date;
```

12. List the event(s) with the highest number of sessions.

```
SELECT e.title, COUNT(s.session_id) AS total_sessions
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(s.session_id) = (
    SELECT MAX(cnt)
    FROM (
        SELECT COUNT(*) AS cnt
        FROM Sessions
        GROUP BY event_id
    ) x
);
```

13. Calculate the average feedback rating of events conducted in each city.

```
SELECT e.city, AVG(f.rating) AS avg_rating
FROM Events e
```

```
JOIN Feedback f ON e.event_id = f.event_id  
GROUP BY e.city;
```

14. List top 3 events based on total registrations.

```
SELECT e.title, COUNT(r.registration_id) AS total_registrations  
FROM Events e  
JOIN Registrations r ON e.event_id = r.event_id  
GROUP BY e.event_id, e.title  
ORDER BY total_registrations DESC  
LIMIT 3;
```

15. Identify overlapping sessions within the same event.

```
SELECT s1.event_id, s1.title AS session1, s2.title AS session2  
FROM Sessions s1  
JOIN Sessions s2  
ON s1.event_id = s2.event_id  
AND s1.session_id < s2.session_id  
AND s1.start_time < s2.end_time  
AND s1.end_time > s2.start_time;
```

16. Find users who created an account in the last 30 days but haven't registered for any events.

```
SELECT u.*  
FROM Users u  
LEFT JOIN Registrations r ON u.user_id = r.user_id  
WHERE u.registration_date >= CURDATE() - INTERVAL 30 DAY  
AND r.registration_id IS NULL;
```

17. Identify speakers who are handling more than one session.

```
SELECT speaker_name, COUNT(*) AS total_sessions  
FROM Sessions  
GROUP BY speaker_name  
HAVING COUNT(*) > 1;
```

18. List all events that do not have any resources uploaded.

```
SELECT e.title  
FROM Events e  
LEFT JOIN Resources r ON e.event_id = r.event_id  
WHERE r.resource_id IS NULL;
```

19. For completed events, show total registrations and average feedback rating.

```
SELECT e.title,  
       COUNT(DISTINCT r.registration_id) AS registrations,  
       AVG(f.rating) AS avg_rating  
FROM Events e  
LEFT JOIN Registrations r ON e.event_id = r.event_id  
LEFT JOIN Feedback f ON e.event_id = f.event_id  
WHERE e.status = 'completed'  
GROUP BY e.event_id, e.title;
```

20. For each user, calculate events attended and feedbacks submitted.

```
SELECT u.full_name,  
       COUNT(DISTINCT r.event_id) AS events_attended,  
       COUNT(DISTINCT f.feedback_id) AS feedbacks_submitted  
FROM Users u  
LEFT JOIN Registrations r ON u.user_id = r.user_id  
LEFT JOIN Feedback f ON u.user_id = f.user_id  
GROUP BY u.user_id, u.full_name;
```

21. List top 5 users who submitted the most feedback entries.

```
SELECT u.full_name,  
       COUNT(f.feedback_id) AS feedback_count  
FROM Users u  
JOIN Feedback f ON u.user_id = f.user_id  
GROUP BY u.user_id, u.full_name  
ORDER BY feedback_count DESC  
LIMIT 5;
```

22. Detect if a user has been registered more than once for the same event.

```
SELECT user_id, event_id, COUNT(*) AS duplicate_count  
FROM Registrations  
GROUP BY user_id, event_id  
HAVING COUNT(*) > 1;
```

23. Show a month-wise registration count trend over the past 12 months.

```
SELECT YEAR(registration_date) AS year,  
       MONTH(registration_date) AS month,  
       COUNT(*) AS registrations  
FROM Registrations
```

```
WHERE registration_date >= CURDATE() - INTERVAL 12 MONTH
GROUP BY YEAR(registration_date), MONTH(registration_date)
ORDER BY year, month;
```

24. Compute the average duration (in minutes) of sessions in each event.

```
SELECT e.title,
       AVG(TIMESTAMPDIFF(MINUTE, s.start_time, s.end_time))
       AS avg_duration_minutes
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title;
```

25. List all events that currently have no sessions scheduled under them.

```
SELECT e.title
FROM Events e
LEFT JOIN Sessions s ON e.event_id = s.event_id
WHERE s.session_id IS NULL;
```